

作业 4：SM3 软件实现与双架构 SIMD 优化

x86 AVX2/AVX-512 与 ARM64 NEON 的多消息并行

段沛旭 董胥宏 靳博涵

摘要

本实验实现 SM3 标量版本, 并采用“SIMD 寄存器负责消息扩展和压缩状态、通用寄存器负责地址循环和常量控制”的混合方式, 在 x86-64 实现 AVX2/AVX-512F, 在 ARM64 实现 NEON。不同消息的同名状态字被放入向量 lane, 实现 8/16/4 路跨消息并行。所有路径通过 abc 标准向量和多个边界长度差分; x86 AVX2 为标量 7.41 倍, ARM NEON 为同平台 2.67 倍。

关键词: SM3; AVX2; AVX-512; NEON; SIMD; multi-buffer

目录

摘要	1
1 实验目标、平台与实现矩阵	2
2 SM3 总体结构与初始向量	2
3 消息填充与边界长度	2
4 消息扩展 W 与 W'	3
5 压缩函数与轮函数	3
6 为什么采用跨消息 SIMD	4
7 x86 AVX2 八路实现	4
8 x86 AVX-512F 十六路实现	4
9 ARM64 NEON 四路实现	5
10 统一接口、调度与尾批	5
11 正确性与边界差分	6
12 性能结果与加速比	7
13 反汇编验证、限制与改进	7
14 复现、结论与参考资料	8

第 1 节 实验目标、平台与实现矩阵

目标是在保持标准语义的前提下完成标量、x86 AVX2/AVX-512F、ARM64 NEON, 并通过统一接口自动选择可用实现。x86 路径需要运行时检测, ARM64 Advanced SIMD 为基础环境。

性能数据来自 x86-64 与阿里云倚天 710 ARM 实例。吞吐测试使用 16 条、每条 4096 bytes 消息并重复 2000 轮, 评价批处理吞吐而非单条短消息延迟。

实现	向量宽度	lane	平台
Scalar	32	1	通用
AVX2	256	8	x86-64
AVX-512F	512	16	x86-64 支持时
NEON	128	4	ARM64

第 2 节 SM3 总体结构与初始向量

SM3 输出 256-bit, 消息按 512-bit 分组。初始向量为 8 个 32-bit 常量, 逐分组执行压缩 $V_{i+1} = CF(V_i, B_i)$, 最后按大端拼接 $A \cdots H$ 。

与 Merkle-Damgård 结构一致, 分组顺序存在串行依赖; 但同一批独立消息之间没有依赖, 这正是 multi-buffer SIMD 的并行来源。



图 1: SM3 从消息到 256-bit 摘要的处理链

第 3 节 消息填充与边界长度

填充先附加 1 bit, 再加 k 个 0, 使长度满足 $l + 1 + k \equiv 448 \pmod{512}$, 最后附加 64-bit 大端原始长度。55/56 与 63/64 bytes 是关键边界: 前者决定是否新增分组, 后者

考验整块处理。

测试集合包含 0、1、3、55、56、63、64、65、127、128、129、1024 bytes。每个 SIMD lane 的结果与标量逐字节比较，可发现尾批、长度编码和多分组链接错误。

```
1 append 0x80
2 while (len % 64) != 56:
3     append 0x00
4 append uint64_be(original_bit_length)
```

第 4 节 消息扩展 W 与 W'

16 个输入字扩展为 $W_0 \cdots W_{67}$:

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus ROTL^{15}(W_{j-3})) \oplus ROTL^7(W_{j-13}) \oplus W_{j-6}$$

随后 $W'_j = W_j \oplus W_{j+4}$ 。

SIMD 版本让每个 `__m256i/uint32x4_t` 保存多条消息的同一 W_j ，循环移位、异或和 P1 全部逐 lane 完成。输入转置是额外成本，但在长批量中被 64 轮计算摊薄。

对象	数量	作用
W	68×32 bit	轮输入与状态更新 与 A/B/C/D 路径混合 扩展扩散
W'	64×32 bit	
P1	线性置换	

第 5 节 压缩函数与轮函数

每轮计算：

$$SS1 = ROTL^7(ROTL^{12}(A) + E + ROTL^j(T_j))$$

$$SS2 = SS1 \oplus ROTL^{12}(A)$$

再得到 TT1、TT2。0-15 轮使用异或型 FF/GG，16-63 轮切换为多数/选择函数。

所有加法按 2^{32} 模进行，循环移位必须避免 C 中移位 32 的未定义行为。最后将 $A \cdots H$ 与输入链接值异或，而不是覆盖。

```
1 TT1 = FF(A,B,C,j) + D + SS2 + Wp[j];
2 TT2 = GG(E,F,G,j) + H + SS1 + W[j];
3 D=C; C=ROTL9(B); B=A; A=TT1;
4 H=G; G=ROTL19(F); F=E; E=P0(TT2);
```

第 6 节 为什么采用跨消息 SIMD

单消息的 64 轮状态依赖导致轮内并行空间有限；跨消息方法把 8/16/4 条独立消息的 A 放在一个向量、B 放在另一个向量，同一条 intrinsic 同时更新所有 lane。

该模型特别适合文件分块、证书批验、数据库哈希与服务端请求聚合。若只有一条消息或消息长度差异很大，lane 利用率下降，需要调度层按长度分桶。



图 2: 同名状态字跨消息装入 SIMD lane

第 7 节 x86 AVX2 八路实现

AVX2 使用 `__m256i` 表示 $8 \times 32\text{-bit}$ 。XOR/AND/OR/ADD 映射到 `_mm256_*`，循环移位由 `slli/srli` 与 OR 组合。消息扩展和压缩的向量语义与标量逐句对应。

入口用 CPU feature 检测避免旧处理器执行非法指令。编译器 target 属性把 AVX2 函数与通用调度分离，使同一二进制可保留标量回退。

- 8 条等长消息形成一个完整 batch。
- 尾部不足 8 条时回退标量或复制安全缓冲。
- 对齐不是正确性要求，但会影响装载效率。

第 8 节 x86 AVX-512F 十六路实现

AVX-512F 把状态扩为 `__m512i`，一批 16 条消息。代码结构与 AVX2 相同，但寄存器更宽、寄存器数量更多；理论吞吐更高，同时可能触发频率下降。

正式部署需分别测量 AVX2 与 AVX-512，而不是总选最宽。Windows/MinGW 对 AVX-512 异常展开可能不兼容，课程复现建议 Linux GCC/Clang。

比较	AVX2	AVX-512F
lane	8	16
寄存器宽度	256 bit	512 bit
部署检测	CPUID	CPUID+OSXSAVE
风险	尾批浪费	降频/工具链兼容

第 9 节 ARM64 NEON 四路实现

NEON 使用 uint32x4_t 保存 4 路状态,veorq_u32,vaddq_u32,vshlq_n_u32/vshrq_n_u32 组合轮函数。AArch64 环境保证 Advanced SIMD 可用，因此调度比 x86 简单。
倚天 710 还报告 SVE/SVE2 与 SM3 专用指令，但本作业聚焦 SIMD+ 通用寄存器混合，所以采用通用 NEON 多消息实现，便于与 AVX2 对照。

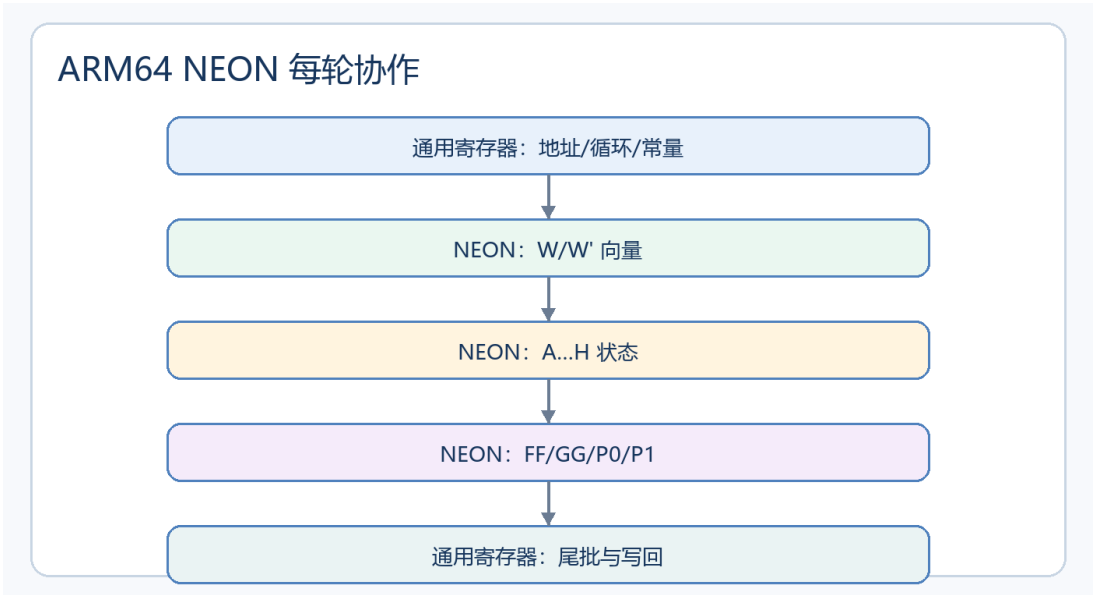


图 3: SIMD 数据通路与通用寄存器控制通路

第 10 节 统一接口、调度与尾批

sm3_hash_many(impl, inputs, n, len, outputs) 是统一入口。调用方选择实现或使用可用性查询；测试程序提供 hash、compare、bench 子命令。
当前同一批要求消息长度相同。n 不是 lane 整数倍时必须避免读取不存在指针；实现可对完整 batch 使用 SIMD，对剩余消息使用标量。统一接口让同一套测试验证所有架构。

1

./sm3_demo

```

2 ./sm3_demo hash avx2 abc
3 ./sm3_demo compare 'custom text'
4 ./sm3_demo bench

```

第 11 节 正确性与边界差分

abc 标准摘要为:

```

1 66c7f0f462eeedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0

```

x86 标量/AVX2 与 ARM 标量/NEON 全部 PASS。

随机缓冲按 lane 使用不同模式填充, 再对 12 个边界长度逐 lane 调用标量复核。这样既验证 SIMD 算术, 也验证转置、分组循环与输出拆分。

SM3 标准向量与边界长度差分测试

```

75 printf(" result      : %s\n",memcmp(scalar,out[0],32)?"FAIL":"MATCH (PASS)");
76 return memcmp(scalar,out[0],32)!=0;
77 }
78
79 static int test_impl(sm3_impl impl)
80 {
81     static const char expect[]="66c7f0f462eeedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0";
82     static const size_t lens[]={0,1,3,55,56,63,64,65,127,128,129,1024};
83     const uint8_t *p[16];uint8_t out[16][32],ref[32],buf[16][1024];char s[65];
84     size_t i,k,n=impl==SM3_IMPL_AVX512?16:impl==SM3_IMPL_AVX2?8:impl==SM3_IMPL_NEON?4:1;
85     for(i=0;i<n;++i)p[i]=(const uint8_t*)"abc";
86     if(sm3_hash_many(impl,p,n,3,out))return 0;
87     for(i=0;i<n;++i){hex(s,out[i]);if(strcmp(s,expect))return 0;}
88     for(i=0;i<n;++i){for(k=0;k<sizeof(buf[i]);++k)buf[i][k]=(uint8_t)(i*29+k*131+7);p[i]=buf[i];}
89     for(k=0;k<sizeof(lens)/sizeof(lens[0]);++k){
90         if(sm3_hash_many(impl,p,n,lens[k],out))return 0;
91         for(i=0;i<n;++i){sm3_hash(p[i],lens[k],ref);if(memcmp(ref,out[i],32))return 0;}
92     }
93     return 1;
94 }
95
96 static void benchmark(sm3_impl impl)
97 {
98     enum{N=16,L=4096,ROUNDS=2000};uint8_t *buf=malloc(N*L),out[N][32];const uint8_t*p[N];clock_t a,b;int i,r;
99     if(!buf)return;
100     for(i=0;i<N*L;++i)buf[i]=(uint8_t)(i*131+17);
101     for(i=0;i<N;++i)p[i]=buf+i*L;
102     a=clock();for(r=0;r<ROUNDS;++r)sm3_hash_many(impl,p,N,L,out);b=clock();
103     sec=(double)(b-a)/CLOCKS_PER_SEC;mb=(double)N*L*ROUNDS/1000000.0;
104     printf(" %-7s %8.2f MB/s  (%.1f MB, %.3f s)\n",sm3_impl_name(impl),mb/sec,mb,sec);free(buf);

```

图 4: 标准向量和多边界长度差分测试代码

第 12 节 性能结果与加速比

x86 标量 192.19 MB/s, AVX2 1424.70 MB/s, 加速 $7.41\times$, 接近 8 lane 理论宽度。ARM64 倚天 710 标量 109.68, NEON 293.19, 加速 $2.67\times$ 。

NEON 未达 $4\times$ 主要因为输入转置、向量构造、尾部填充、输出拆分与较短消息的固定开销; AVX2 结果说明长等长批次能较充分摊薄这些成本。

平台	实现	吞吐 (MB/s)	加速比
x86-64	标量	192.19	$1.00\times$
x86-64	AVX2	1424.70	$7.41\times$
ARM64	标量	109.68	$1.00\times$
ARM64	NEON	293.19	$2.67\times$

SM3 跨消息 SIMD 吞吐

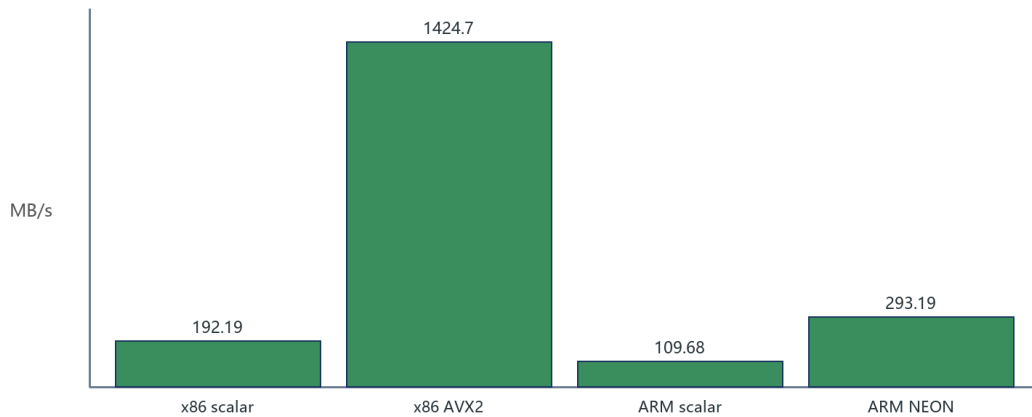


图 5: x86 与 ARM 平台的同平台标量/SIMD 吞吐

第 13 节 反汇编验证、限制与改进

源码使用 intrinsic 不等于最终真的执行向量指令。ARM 实验用 objdump 确认 add/eor/ushr 等 NEON 指令; x86 应确认 vpaddd/vpxor/vpslld 等, 并检查热循环没有意外标量化。

限制包括等长批次、单线程、无 SVE2/SM3 扩展、未对不同 batch size 画吞吐曲线。后续可增加长度分桶、线程池、ARMv8.2-SM3 专用指令和 AVX-512 频率对照。

- 性能结果只代表指定 CPU/编译器。
- 单消息延迟可能不受益。
- 必须保留标量路径作为可信参考与兼容回退。

第 14 节 复现、结论与参考资料

Linux/x86 运行 make 后执行 compare 与 bench; ARM64 使用同一 Makefile, NEON 文件在 AArch64 编译。复现应保存 lscpu、gcc -version、CFLAGS 和反汇编片段。

实验完成题目要求的两类架构指令集, 并证明跨消息 SIMD 能显著提高 SM3 批处理吞吐。正确性覆盖标准向量和关键填充边界, 性能结果与理论 lane 宽度关系清晰。